HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÀI TẬP LỚN

MÔN HỌC: LẬP TRÌNH PYTHON

Giảng viên hướng dẫn: **Lê Văn Vịnh**

Nhóm: **05**

Lớp: **D24CQCE05–B**

Họ tên	MSV
Đặng Đức Anh	B24DCCE005
Nguyễn Duy Khương	B24DCCE152
Nguyễn Minh Đức	B24DCCE054

Hà Nội,2026

Bài Tập Lập Trình Với Python

I.

Viết chương trình Python thu thập dữ liệu phân tích cầu thủ với yêu cầu như sau:

I.1

- Thu thập dữ liệu thống kê của tất cả các cầu thủ có số phút thi đấu nhiều hơn 90 phút tại giải bóng đá ngoại hạng Anh mùa 2024-2025.
- Nguồn dữ liệu: https://fbref.com/en/
- Ghi kết quả vào một bảng trong cơ sở dữ liệu SQLite, bảng kết quả có cấu trúc như sau:
    + Mỗi cột tương ứng với một chỉ số.
    + Chỉ số nào không có hoặc không áp dụng thì để là N/a
    + Tham khảo: https://fbref.com/en/squads/822bd0ba/2023-2024//en/squads/822bd0ba/2023-2024/Liverpool-Stats

I.2

- Với mỗi cầu thủ được ghi trong bảng trên, thu thập giá chuyển nhượng của cầu thủ trong mùa 2024-2025 từ trang web https://www.footballtransfers.com. Nếu không có dữ liệu thì để là N/a
- Kết quả ghi vào một bảng thứ hai trong cơ sở dữ liệu SQLite.

Chú ý:

Trong khi crawl dữ liệu, có thể gặp các tình huống như Capchar và hoặc rate-limit. Đề xuất phương pháp khắc phục nếu có

Có thể dùng module request hoặc selenium để cào dữ liệu.

II.

II.1

Sử dụng module Python Flask để tạo các Restful API tra cứu chỉ số cầu thủ trong cơ sở dữ liệu vừa tạo ở câu 1 theo các tiêu chí sau:

- Tên cầu thủ: trả về toàn bộ chỉ số của cầu thủ có tên tương ứng.
- Câu lạc bộ: Trả về toàn bộ chỉ số của các cầu thủ thuộc về câu lạc bộ đó.

II.2

Sử dụng module Request, viết 1 chương trình python để tra cứu dữ liệu cầu thủ theo 2 tiêu chí như trên. Tên cầu thủ hoặc câu lạc bộ được truyền vào lúc chạy chương trình có cú pháp như sau:

python lookup.py --name <tên cầu thủ> --club <tên câu lạc bộ>

- Kết quả trả ra được in ra màn hình dưới dạng bảng, và 1 file CSV. Tên file được đặt theo tên cầu thủ hoặc câu lạc bộ, tùy theo sử dụng tham số đầu vào.

III.

III.1

- Tìm trung vị, trung bình và độ lệch chuẩn của mỗi chỉ số các cầu thủ của mỗi đội. Ghi kết quả vào 1 file CSV.
- Tìm đội bóng có chỉ số điểm số cao nhất ở mỗi chỉ số. Theo bạn đội nào có phong độ tốt nhất giải ngoại Hạng Anh mùa 2024-2025

III.2

- Dựa vào kết quả từ câu I, đề xuất 1 phương pháp định giá cầu thủ.

III.

- Sử dụng thuật toán K-means để phân loại các cầu thủ thành các nhóm có chỉ số tương tự nhau.
- Theo bạn thì nên phân loại cầu thủ thành bao nhiêu nhóm? Vì sao (gợi ý: sử dụng biểu đồ Elbow và Silhoute, vẽ 2 biểu đồ này)? Bạn có Nhận xét gì về kết quả.

\- Sử dụng thuật toán PCA, giảm số chiều dữ liệu xuống 2 chiều, 3 chiều, vẽ hình scatter plot phân cụm các điểm dữ liệu trên mặt 2D, và khối 3D.

Báo cáo bài tập

Bài I:

A. Yêu cầu bài tập:

I.

Viết chương trình Python thu thập dữ liệu phân tích cầu thủ với yêu cầu như sau:

I.1

- Thu thập dữ liệu thống kê của tất cả các cầu thủ có số phút thi đấu nhiều hơn 90 phút tại giải bóng đá ngoại hạng Anh mùa 2024-2025.
- Nguồn dữ liệu: https://fbref.com/en/
- Ghi kết quả vào một bảng trong cơ sở dữ liệu SQLite, bảng kết quả có cấu trúc như sau:
  + Mỗi cột tương ứng với một chỉ số.
  + Chỉ số nào không có hoặc không áp dụng thì để là N/a

+ Tham khảo:
https://fbref.com/en/squads/822bd0ba/2023-2024//en/squads/822bd0ba/2023-2024/Liverpool-Stats

I.2

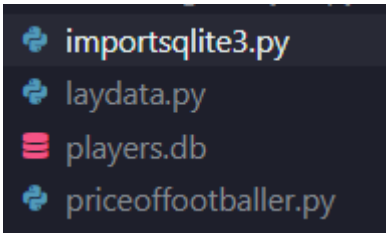
- Với mỗi cầu thủ được ghi trong bảng trên, thu thập giá chuyển nhượng của cầu thủ trong mùa 2024-2025 từ trang web https://www.footballtransfers.com. Nếu không có dữ liệu thì để là N/a
- Kết quả ghi vào một bảng thứ hai trong cơ sở dữ liệu SQLite.

Chú ý:

Trong khi crawl dữ liệu, có thể gặp các tình huống như Captcha và hoặc rate-limit. Đề xuất phương pháp khắc phục nếu có

Có thể dùng module request hoặc selenium để cào dữ liệu.

B. Cấu trúc chương trình:



1. **laydata.py:** crawl dữ liệu các cầu thủ theo yêu cầu từ trang https://fbref.com/en/, trả về 1 dictionary các tuyển thủ
2. **importsqlite3.py:**từ dữ liệu crawl được, tạo bảng database các cầu thủ bằng sqlite3.
3. **priceoffootballer.py:**dựa vào tên các cầu thủ có trong database, tra cứu từng tên trên https://www.footballtransfers.com để tìm giá trị cầu thủ hiện tại, điền vào bảng thứ 2 trong players.db

C. Giải thích chi tiết:

a. Crawl dữ liệu từ https://fbref.com/en/(laydata.py) :

i.

```
from seleniumbase import SB
from bs4 import BeautifulSoup
```

1. **dùng seleniumbase để giải quyết vấn đề captcha**

2. dùng beautifulsoup để parse bảng tĩnh các cầu thủ trong trang web

Player Standard Stats 2024-2025 Premier League ☑ Hide non-qualifiers for rate stats Glossary Toggle Per90 Stats

							Playing Time				Performance								Per 90 Minutes					
Rk	Player	Nation	Pos	Squad	Age	Born	MP	Starts	Min	90s	Gls	Ast	G+A	G-PK	PK	PKatt	CrdY	CrdR	Gls	Ast	G+A	G-PK	G+A-PK	Matches
1	Max Aarons	ENG	DF	Bournemouth	24	2000	3	1	86	1.0	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
2	Joshua Acheampong	ENG	DF	Chelsea	18	2006	4	2	170	1.9	0	0	0	0	0	0	1	0	0.00	0.00	0.00	0.00	0.00	Matches
3	Tyler Adams	USA	MF	Bournemouth	25	1999	28	21	1,965	21.8	0	3	3	0	0	0	7	0	0.00	0.14	0.14	0.00	0.14	Matches
4	Tosin Adarabioyo	ENG	DF	Chelsea	26	1997	22	15	1,409	15.7	1	1	2	1	0	0	4	0	0.06	0.06	0.13	0.06	0.13	Matches
5	Simon Adingra	CIV	MF	Brighton	22	2002	29	12	1,097	12.2	2	2	4	2	0	0	0	0	0.16	0.16	0.33	0.16	0.33	Matches
6	Emmanuel Agbadou	CIV	DF	Wolves	27	1997	16	16	1,410	15.7	1	0	1	1	0	0	3	0	0.06	0.00	0.06	0.06	0.06	Matches
7	Asher Agbinone	ENG	MF	Crystal Palace	18	2005	2	0	6	0.1	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
8	Ola Aina	NGA	DF	Nottingham Forest	27	1996	35	35	2,995	33.3	2	1	3	2	0	0	5	0	0.06	0.03	0.09	0.06	0.09	Matches
9	Rayan Aït-Nouri	ALG	MF,DF	Wolves	23	2001	37	37	3,109	34.5	4	7	11	4	0	0	7	1	0.12	0.20	0.32	0.12	0.32	Matches
10	Kristoffer Ajer	NOR	DF	Brentford	26	1998	24	17	1,438	16.0	0	0	0	0	0	0	5	0	0.00	0.00	0.00	0.00	0.00	Matches
11	Manuel Akanji	SUI	DF,MF	Manchester City	29	1995	26	23	2,014	22.4	0	0	0	0	0	0	3	0	0.00	0.00	0.00	0.00	0.00	Matches
12	Nathan Aké	NED	DF	Manchester City	29	1995	10	8	678	7.5	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
13	Carlos Alcaraz	ARG	MF	Southampton	21	2002	1	0	10	0.1	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
14	Carlos Alcaraz	ARG	MF	Everton	21	2002	15	7	764	8.5	2	3	5	2	0	0	4	0	0.24	0.35	0.59	0.24	0.59	Matches
15	Trent Alexander-Arnold	ENG	DF	Liverpool	25	1998	33	28	2,365	26.3	3	6	9	3	0	0	5	0	0.11	0.23	0.34	0.11	0.34	Matches
16	Alisson	BRA	GK	Liverpool	31	1992	28	28	2,508	27.9	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
17	Miguel Almirón	PAR	FW	Newcastle United	30	1994	9	1	157	1.7	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
18	Olabade Aluko		DF	Leicester City			1	0	2	0.0	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
19	Edson Álvarez	MEX	MF	West Ham United	26	1997	28	20	1,772	19.7	0	1	1	0	0	0	9	1	0.00	0.05	0.05	0.00	0.05	Matches
20	Will Alves	ENG	MF	Leicester City	19	2005	1	0	1	0.0	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
21	Harry Amass	ENG	MF	Manchester Utd	17	2007	5	4	346	3.8	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
22	Samuel Amo-Ameyaw	ENG	FW,MF	Southampton	18	2006	2	0	28	0.3	0	0	0	0	0	0	1	0	0.00	0.00	0.00	0.00	0.00	Matches
23	Mathis Amougou	FRA	MF	Chelsea	18	2006	1	0	8	0.1	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
24	Joachim Andersen	DEN	DF	Crystal Palace	28	1996	1	1	90	1.0	0	0	0	0	0	0	1	0	0.00	0.00	0.00	0.00	0.00	Matches
25	Joachim Andersen	DEN	DF	Fulham	28	1996	29	29	2,583	28.7	0	0	0	0	0	0	6	1	0.00	0.00	0.00	0.00	0.00	Matches
Rk	Player	Nation	Pos	Squad	Age	Born	MP	Starts	Min	90s	Gls	Ast	G+A	G-PK	PK	PKatt	CrdY	CrdR	Gls	Ast	G+A	G-PK	G+A-PK	Matches
26	Elliot Anderson	ENG	MF	Nottingham Forest	21	2002	37	33	2,728	30.3	2	6	8	2	0	0	10	0	0.07	0.20	0.26	0.07	0.26	Matches
27	André	BRA	MF	Wolves	23	2001	33	31	2,472	27.5	0	0	0	0	0	0	7	0	0.00	0.00	0.00	0.00	0.00	Matches
28	Michail Antonio	JAM	FW	West Ham United	34	1990	14	11	836	9.3	1	1	2	1	0	0	1	0	0.11	0.11	0.22	0.11	0.22	Matches
29	Antony	BRA	FW,MF	Manchester Utd	24	2000	8	0	141	1.6	0	0	0	0	0	0	0	0	0.00	0.00	0.00	0.00	0.00	Matches
30	Julian Araujo	MEX	DF	Bournemouth	22	2001	12	7	497	5.5	0	0	0	0	0	0	2	0	0.00	0.00	0.00	0.00	0.00	Matches
31	Cameron Archer	ENG	FW	Southampton	23	2001	35	13	1,441	16.0	2	0	2	2	0	1	0	0	0.12	0.00	0.12	0.12	0.12	Matches

ii. Mảng các cột cần crawl trong bảng:

```
stats = [
    "player",
    "nationality",
    "position",
    "team",
    "age",
    "birth_year",
    "games",
    "games_starts",
    "minutes",
    "goals",
    "assists",

    "goals_assists_pens_per90",
    "goals_pens_per90",
    "goals_assists_per90",
    "assists_per90",
    "goals_per90",
    "cards_red",
    "cards_yellow",
    "pens_att",
    "pens_made",
    "goals_pens",
    "goals_assists",
    "minutes_90s"
]
```

iii. xóa dấu “,” trong số phút để xét số phút lớn hơn 90:

```python
def fixmin(s):
    if "," in s:
        return s.replace(",","")
    return s
```

iv. Đẩy dữ liệu:

```python
def get_players():
    ans=[]
    with SB(uc=True) as sb:
        sb.open("https://fbref.com/en/comps/9/2024-2025/stats/2024-2025-Premier-League-Stats")

        sb.sleep(20)  # đảm bảo load xong

        html = sb.get_page_source()
        #html = re.sub(r"<!--|-->", "", html)

        soup = BeautifulSoup(html, "html.parser")

        table = soup.find("table", {"id": "stats_standard"})
        cauthus=table.select("tbody tr")

        for cauthu in cauthus:
            player={}
            for stat in stats:
                hang=cauthu.find("td",{"data-stat":stat})
                if hang:
                    for icon in hang.find_all("span",class_=lambda x:x and "f-" in x):
                        icon.decompose()
                    player[stat]=hang.text.strip()
                else:
                    player[stat]="N/A"
            if player["minutes"] != "N/A" and int(fixmin(player["minutes"]))>90:
                ans.append(player)
    return ans
```

1. kết nối và parse html vào beautiful soup:

```python
def get_players():
    ans=[]
    with SB(uc=True) as sb:
        sb.open("https://fbref.com/en/comps/9/2024-2025/stats/2024-2025-Premier-League-Stats")

        sb.sleep(20)  # đảm bảo load xong

        html = sb.get_page_source()
        #html = re.sub(r"<!--|-->",

        soup = BeautifulSoup(html, "html.parser")
```

```
(module) html
General functions for HTML manipulation.
```

   a. để đảm bảo selenium đã vượt qua được captcha, cần chờ(sleep(20s)) để html của web được load xong
   b. sau đó lấy source của toàn bộ trang đã load, dùng soup để tra và parse dữ liệu bảng cần thiết.

2. sử dụng soup, parse từng cầu thủ:

```
    table = soup.find("table", {"id": "stats_standard"})
    cauthus=table.select("tbody tr")

    for cauthu in cauthus:
        player={}
        for stat in stats:
            hang=cauthu.find("td",{"data-stat":stat})
            if hang:
                for icon in hang.find_all("span",class_=lambda x:x and "f-" in x):
                    icon.decompose()
                player[stat]=hang.text.strip()
            else:
                player[stat]="N/A"
        if player["minutes"] != "N/A" and int(fixmin(player["minutes"]))>90:
            ans.append(player)
return ans
```

a.
   - tìm bảng có id= stats_standard (là bảng dữ liệu các cầu thủ)
   - gắn tất cả các nhãn tbody và tr vào cauthus (các nhãn này chứa đầy đủ thông tin của từng cầu thủ)

b.
   - duyệt từng cầu thủ
   - duyệt từng chỉ số cần tìm: tìm trong cầu thủ đó nhãn td có mục "data-stat"= với stat cần tìm, trả về biến hang
   - nếu hang tìm ra được giá trị: xóa hết các icon, khoảng trống thừa trước và sau text chứa dữ liệu đó.Sau đó thêm vào dictionary player(dictionary chứa dữ liệu cầu thủ) với cặp key value là: stat,dữ liệu
   - nếu hàng không tìm được nhãn: thêm vào dictionary player với cặp key value là: stat,"N/A".
   - nếu các cầu thủ có dữ liệu thời gian thi đấu hoặc dữ liệu thời gian thi đấu >90 phút: thêm dictionary player đó vào mảng kết quả
   - lặp lại cho tất cả cầu thủ

➢ kết quả stat của các cầu thủ được lưu vào mảng ans.

b. tạo database(importsqlite3.py):

1. tạo bảng database rỗng dựa trên các stat:

```python
import sqlite3
from laydata import get_players

# tạo bảng
def create_table():
    conn = sqlite3.connect("players.db")
    cursor = conn.cursor()
    cursor.execute("DROP TABLE IF EXISTS players")
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS players (
        player TEXT,
        nationality TEXT,
        position TEXT,
        team TEXT,
        age TEXT,
        birth_year TEXT,
        games TEXT,
        games_starts TEXT,
        minutes INTEGER,
        goals TEXT,
        assists TEXT,
        goals_assists_pens_per90 TEXT,
        goals_pens_per90 TEXT,
        goals_assists_per90 TEXT,
        assists_per90 TEXT,
        goals_per90 TEXT,
        cards_red TEXT,
        cards_yellow TEXT,
        pens_att TEXT,
        pens_made TEXT,
        goals_pens TEXT,
        goals_assists TEXT,
        minutes_90s TEXT
    )
    """)

    conn.commit()
    conn.close()
```

2. nhập dữ liệu từ mảng vào database:

```python
# insert dữ liệu
def insert_data(players):
    conn = sqlite3.connect("players.db")
    cursor = conn.cursor()

    for p in players:
        cursor.execute("""
        INSERT INTO players VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
        """, (
            p["player"], p["nationality"], p["position"], p["team"],
            p["age"], p["birth_year"], p["games"], p["games_starts"],
            int(p["minutes"].replace(",", "")),
            p["goals"], p["assists"],
            p["goals_assists_pens_per90"], p["goals_pens_per90"],
            p["goals_assists_per90"], p["assists_per90"], p["goals_per90"],
            p["cards_red"], p["cards_yellow"],
            p["pens_att"], p["pens_made"],
            p["goals_pens"], p["goals_assists"],
            p["minutes_90s"]
        ))

    conn.commit()
    conn.close()
```

- kết nối với players.db, insert từng stat của từng player vào bảng đã kẻ sẵn ở trên

3. chạy main:

```python
if __name__ == "__main__":

    create_table()
    data = get_players()
    insert_data(data)
    print("Đã lưu dữ liệu vào SQLite!")
```

- tạo bảng (hàm create_table)
- lấy mảng data bằng hàm crawl từ laydata.py
- đẩy dữ liệu từ mảng vào bảng players.db
- in ra tin nhắn hoàn thành chương trình.

➢ **kết quả:1 bảng thứ 1 trong database player.db tên players, có tên cầu thủ thi đấu hơn 90 phút ở premier league, các chỉ số theo trang https://www.footballtransfers.com.**

c. tra cứu giá cầu thủ(priceoffootballer.py):

i. import thư viện , kết nối database,lấy tên cầu thủ:

```
conn=sqlite3.connect('players.db')

curs=conn.cursor()
curs.execute("DROP TABLE if EXISTS prices")

curs.execute("""
    CREATE TABLE IF NOT EXISTS prices (
        player TEXT,
        price TEXT
    )
""")

curs.execute("SELECT player FROM players")
rows = curs.fetchall()
conn.commit()
```

1. kết nối với players.db.
2. tạo bảng thứ 2 trong db tên là prices, gồm tên và giá trị cầu thủ.
3. cho cursor curs chỏ vào tên của các players,đẩy tất cả tên vào mảng rows (bằng curs.fetchall()).

ii. tra tên trên web, lấy và điền dữ liệu vào database:

```
with SB(uc=True) as sb:
    sb.open("https://www.footballtransfers.com/en/search")

    for row in rows:
        player_name = str(row[0]).strip()
        print(f"Đang tìm giá cho: {player_name}")

        sb.type('input[name="search_value"]', f"{player_name}\n")

        pri = 'N/A'
        try:
            player_link_selector = f'a[title*="{player_name}"]'

            sb.wait_for_element(player_link_selector, timeout=6)

            price_selector = f'{player_link_selector} .player-price .player-tag'

            if sb.is_element_present(price_selector):
                pri = sb.get_text(price_selector).strip()
            else:
                pri = sb.get_text(f'{player_link_selector} .player-price').strip()

        except Exception:
            pri = 'N/A'

        print(f"-> Kết quả: {pri}")

        curs.execute("INSERT INTO prices (player, price) VALUES (?, ?)", (player_name, pri))
        conn.commit()

conn.close()
print("Hoàn thành!")
```

1. khởi động selenium, truy cập vào web https://www.footballtransfers.com

2. duyệt từng hàng(row):
    a. sửa tên chuẩn (bỏ dấu cách thừa trước,sau),in thông báo tìm giá
    b. lệnh cho selenium nhập tên cầu thủ vào search box(là nhãn input có name=search_value) và tìm,đặt giá trị cầu thủ ban đầu là "N/A"

```
sb.type('input[name="search_value"]', f"{player_name}\n")

pri = 'N/A'
```

    c. thử:
        - sau lệnh **sb.type**, web chuyển sang trang tìm kiếm các cầu thủ có cùng tên.
        - biến tên thành dạng nhãn a có title = tên cầu thủ để tìm kiếm trong html của trang tìm kiếm
        - **sb.wait_for_element**: chờ trang web load xong và có nhãn cầu thủ như dạng đã biến đổi cần tìm, không thì nhảy vào trường hợp except(pri="N/A": không có giá)
        - khi đã tìm được nhãn ghi cầu thủ, tìm sâu vào nhãn có player-price rồi tới player-tag,sau đó lấy text(giá trị cầu thủ).
        - **nếu tìm được đã tìm được giá**, điền giá vào biến pri dưới dạng đã xóa cách trước sau.
        - **không tìm được giá**: đi lại theo đường thẳng vào player-price, không qua player-tag, lấy text và xóa cách trước sau (do 1 vài cầu thủ có cách đặt giá không qua player-tag, mà đặt thẳng ở nhãn player-price).
        - in ra giá trị cầu thủ,điền tên cầu thủ và giá trị mới tìm được vào bảng database.
        - lặp lại với tất cả các cầu thủ.
        - in ra hoàn thành để báo đã nhập xong bảng

➢ **kết quả: 1 bảng thứ 2 trong database player.db tên prices, có tên cầu thủ và giá trị cầu thủ theo trang https://www.footballtransfers.com.**

d. Cách chạy chương trình:

- Chạy file importsqlite3.py để tạo bảng players trong player.db.
- Chạy file priceoffootballer.py để tạo bảng prices trong player.db.

Bài II:

A. Yêu cầu bài tập:

Câu II:

II.1

Sử dụng module Python Flask để tại các Restful API tra cứu chỉ số cầu thủ trong cơ sở dữ liệu vừa tạo ở câu I theo các tiêu chứ sau:

- Tên cầu thủ: Trả lại toàn bộ chỉ số của cầu thủ có tên tương ứng.
- Câu lạc bộ: Trả về toàn bộ chỉ số của các cầu thủ thuộc câu lạc bộ đó.

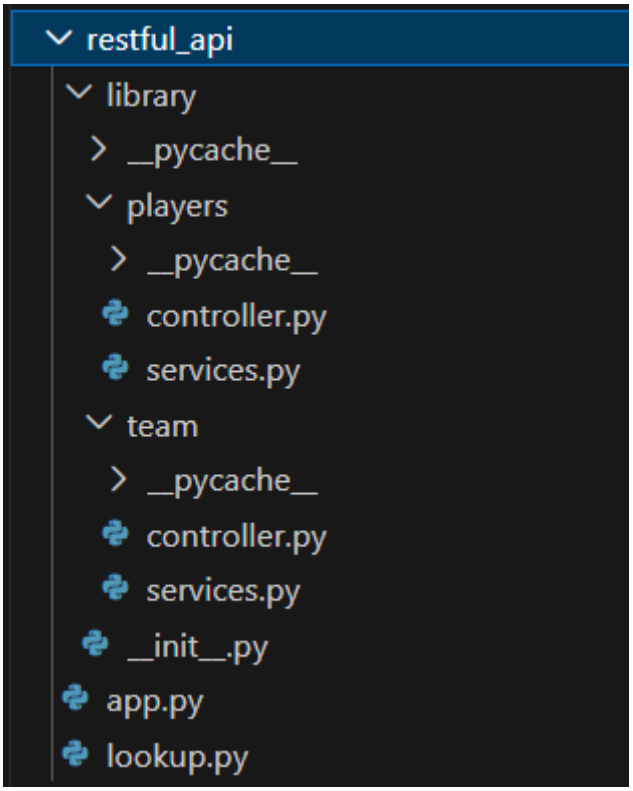
II.2

Sử dụng module Request, viết chương trình Python để tra cứu dữ liệu cầu thủ theo 2 tiêu chí như trên. Tên cầu thủ hoặc được truyền vào lúc chạy chương trình có cú pháp như sau:

python lookup.py –name <tên cầu thủ> –club <tên câu lạc bộ>

Kết quả tra được in ra màn hình dưới dạng bảng, và 1 file CSV. Tên file được đặt theo tên cầu thủ hoặc câu lạc bộ, tùy theo tham số đầu vào.

B. Cấu trúc chương trình:



C. Giải thích chi tiết:

1. **__init__.py:**

```python
from flask import Flask
from .players.controller import player_name as pn
from .team.controller import team_name as tn

def create_app():
    app = Flask(__name__)
    app.json.sort_keys = False

    app.register_blueprint(pn)
    app.register_blueprint(tn)

    return app
```

- from flask import Flask: Thư viện flask cung cấp Flask. Đây là 1 micro web framework nhẹ, rất phù hợp với bài tập yêu cầu tạo các RESTful API.

- from .players.controller import player_name as pn và from .team.controller import team_name as tn: Import 2 phân hệ (Blueprint) player_name (pn) và team_name (tn) từ file controller.py của thư mục players và team. 2 phân hệ này chứa các hàm xử lý logic và đường dẫn (route) riêng biệt để truy vấn theo cầu thủ hoặc câu lạc bộ. Trong hàm create_app(), 2 phân hệ này sẽ được đăng ký vào ứng dụng chính.

- create_app():

+ **app = Flask(__name__):** Khởi tạo ứng dụng. Khi gọi Flask(...), hệ thống đã tạo ra 1 instance thuộc vào class Flask, và app là con trỏ tham chiếu tới instance đó.
+ **__name__** là biến hệ thống của Python. Nó cho hệ thống biết vị trí gốc của gói ứng dụng hiện tại, giúp Flask định vị được tài nguyên nội bộ của ứng dụng đó.
+ **app.json.sort_keys = False:** Khi trả về file JSON, hệ thống mặc định sắp xếp các khóa theo thứ tự bảng chữ cái. Lệnh này sẽ ghi đè lên cấu hình mặc định của JSON, khiến thứ tự các trường dữ liệu được giữ nguyên theo thứ tự đã tạo trong database.
+ **app.register_blueprint(pn) và app.register_blueprint(tn):** 2 câu lệnh này có tác dụng hợp nhất danh sách đường dẫn của pn và tn vào bảng định tuyến (URL Map) của máy chủ (Server). Nếu không có 2 câu lệnh này, server sẽ không biết tới sự tồn tại của các API đã viết, dẫn tới lỗi 404 (NOT FOUND) khi người dùng gửi truy vấn.

+ **return app:** Sau khi đã có đầy đủ cấu hình và chức năng, create_app() sẽ trả về đối tượng app hoàn chỉnh để khởi chạy trong app.py.

2. **app.py:**

```python
from library import create_app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)
```

- from library import create_app: Import hàm create_app() nằm trong thư mục library.
- if __name__ == "__main__": Khi file app.py được chạy trực tiếp từ Terminal, Python sẽ gán cho biến __name__ của file này giá trị là **"__main__".** Câu lệnh này có tác dụng chỉ cho phép khởi chạy nếu app.py là file chạy chính, tránh việc chạy nhầm nếu file bị import vào nơi khác.
- app = create_app(): Sau khi thiết lập đầy đủ cấu hình và chức năng cho đối tượng app trong hàm create_app() (__init__.py), sử dụng con trỏ app để tham chiếu tới đối tượng đó.
- app.run(debug=True): Kích hoạt máy chủ để nhận các truy vấn. Tham số "debug=True" giúp khởi động chế độ gỡ lỗi (tự động tải lại server mỗi khi code thay đổi và hiển thị chi tiết mã lỗi nếu có trên màn hình).

3. **services.py (thư mục players):**

```python
import sqlite3
import os

def get_player_data(name):
    cur_dir = os.path.dirname(os.path.abspath(__file__)) # duong dan services.py
    db_path = os.path.abspath(os.path.join(cur_dir, "../../../players.db")) # tao duong dan toi players.db

    conn = sqlite3.connect(db_path)

    conn.row_factory = sqlite3.Row # dinh hinh ket qua sau truy van
    cursor = conn.cursor()

    query = "SELECT * FROM players WHERE player = ?"
    cursor.execute(query, (name,))

    rows = cursor.fetchall()

    conn.close()

    return [dict(row) for row in rows]
```

- import sqlite3: Thư viện sqlite3 cung cấp các phương thức dùng để mở kết nối và truy vấn trên database (players.db).
- import os: Thư viện os cung cấp các hàm tương tác với hệ điều hành, xử lý và điều hướng file linh hoạt.
- get_player_data():

+ **name:** Tham số đầu vào dùng để nhận tên cầu thủ mà người dùng truy vấn.
+ **cur_dir = os.path.dirname(os.path.abspath(__file__)):** Câu lệnh này có xác định tọa độ tuyệt đối (Absolute Coordinate) của file services.py. Trong đó, "__file__" là biến hệ thống trỏ tới tọa độ hiện tại (có thể thay đổi tùy vào vị trí lưu file trên máy người dùng), os.path.abspath() lấy đường dẫn tuyệt đối, os.path.dirname() trích xuất tên thư mục cha.
+ **db_path = os.path.abspath(os.path.join(cur_dir, "../../player.db")):** Từ tọa độ tuyệt đối gán trong cur_dir, sử dụng os.path.join và lùi lại 3 cấp (../../../) để trỏ chính xác vào players.db. Cách viết này giúp tránh gặp lỗi NOT FOUND khi chạy trên các máy tính hoặc hệ điều hành khác nhau.
+ **conn = sqlite3.connect(db_path):** Khởi tạo 1 phiên làm việc với file players.db thông qua địa chỉ gán ở db_path.
+ **conn.row_factory = sqlite3.Row:** SQLite mặc định trả về kết quả là 1 tuple (không có key → khó truy xuất). Lệnh này ép mỗi hàng được trả về là 1 đối tượng sqlite3.Row (sqlite3.Row object), 1 dạng cấu trúc key-value gần giống dict → dễ dàng hơn khi truy xuất dữ liệu.
+ **cursor = conn.cursor():** Khởi tạo đối tượng con trỏ (cursor object), đóng vai trò trung gian giữa người dùng và hệ thống, nhận lệnh SQL từ Python, mang vào thực thi trong database rồi trả về kết quả.
+ **query = "SELECT * FROM players WHERE player = ?":** Lệnh truy vấn lấy tất cả thông tin của cầu thủ:
    - **SELECT *:** Lấy toàn bộ các trường dữ liệu của bản ghi.
    - **FROM players:** Chỉ lấy ở bảng có tên "players".
    - **WHERE player = ?:** Điều kiện lọc lấy đúng cầu thủ có tên trùng khớp. Việc dùng "?" thay vì nối chuỗi trực tiếp giúp hệ

thống ngăn chặn được lỗ hổng bảo mật tấn công tiêm mã (SQL Injection).

+ **cursor.execute(query, (name,)):** Thực hiện lệnh truy vấn với tham số là "name" được truyền vào (có dấu phẩy ở sau "name" là do hàm execute chỉ nhận vào 1 tuple, nếu thiếu dấu thì hàm sẽ nhận diện đó là dấu ngoặc toán học, dẫn tới lỗi ProgrammingError).
+ **rows = cursor.fetchall():** Lấy toàn bộ bản ghi khớp với điều kiện lưu vào biến rows.
+ **conn.close():** Đóng phiên làm việc với players.db, giải phóng tài nguyên hệ thống cho bộ nhớ.
+ **return [dict(row) for row in rows]:** Sử dụng List Comprehension để duyệt qua từng đối tượng sqlite3.Row trong danh sách, ép kiểu chúng về dict (chuẩn key-value) rồi trả về cho controller.py xử lý.

4. **controller.py (thư mục players):**

```python
from flask import Blueprint, request, jsonify
from .services import get_player_data

player_name = Blueprint("player_name", __name__)
@player_name.route("/get-players/player", methods=["GET"])
def get_players():
    cau_thu = request.args.get("name") # tu khoa flask dung de doi chieu voi phan duoi cua url

    data = get_player_data(cau_thu)

    if not cau_thu:
        return jsonify({
            "message": "vui long nhap ten cau thu"
        }), 400 # loi cu phap

    elif len(data) == 0:
        return jsonify({
            "message": "khong tim thay cau thu nay"
        }), 404 # db khong co cau thu nay hoac duong dan bi nhap sai

    return jsonify({
        "message": "ok",
        "cau thu": data
    }), 200 # da tim thay
```

- **from flask import Blueprint, request, jsonify:** Import các công cụ mà Flask cung cấp. Trong đó Blueprint giúp tổ chức ứng dụng thành các thành phần module (modular) và đăng ký route vào ứng dụng chính, request chứa các phương thức HTTP (HTTP methods), jsonify chuyển đổi kiểu dữ liệu từ dictionary hoặc list trong Python sang chuỗi định dạng chuẩn JSON, đồng thời đóng gói nó vào 1 HTTP Response hợp lệ để gửi về người dùng.

- **from .services import get_player_data:** Import hàm get_player_data() từ file services.py.
- **player_name = Blueprint("player_name", __name__):** Khởi tạo phân hệ mang tên "player_name". __name__ là biến hệ thống của Python, có vai trò xác định tọa độ hiện tại của controller.py.
- **@player_name.route("/get-players/player", methods=["GET"]):** Khai báo bộ định tuyến (Routing). Khi người dùng truy cập vào URL này bằng phương thức "GET" (yêu cầu lấy dữ liệu), hệ thống sẽ tự kích hoạt hàm bên dưới nó.
- **get_players():**
  + **cau_thu = request.args.get("name"):** Là phương thức dùng để trích xuất giá trị từ các tham số của chuỗi truy vấn (query string). "name" là từ khóa giúp Flask đối chiếu với phần đuôi của URL (sau dấu hỏi chấm) để lấy tên cầu thủ.
  + **data = get_player_data(cau_thu):** Truyền từ khóa tìm được vào hàm get_player_data(), chương trình sẽ thực thi lệnh SQL và tìm kiếm tên cầu thủ trùng khớp với "name" trong players.db, sau đó trả về kết quả (được gán vào biến data).
  + Sau khi nhận được kết quả trả về từ services.py, sẽ có **3 trường hợp** xảy ra:
    - **Trường hợp 1 (Lỗi cú pháp - Lỗi 400 Bad Request):** Xảy ra khi người dùng nhập sai từ khóa "name", không nhập tên cầu thủ hoặc nhập thiếu dấu hỏi chấm. Code phát hiện biến "cau_thu" rỗng vã trả về mã lỗi 400, báo cho người dùng biết họ đã gửi 1 yêu cầu không hợp lệ.

```
if not cau_thu:
    return jsonify({
        "message": "vui long nhap ten cau thu"
    }), 400 # loi cu phap
```

    - **Trường hợp 2 (Không tìm thấy - Lỗi 404 NOT FOUND):** Xảy ra khi người dùng nhập sai định dạng URL hoặc cầu thủ họ tìm không có mặt trong players.db. Code trả về mảng data rỗng cùng thông báo không tìm thấy cầu thủ này.

```
elif len(data) == 0:
    return jsonify({
        "message": "khong tim thay cau thu nay"
    }), 404 # db khong co cau thu nay hoac duong dan bi nhap sai
```

- **Trường hợp 3 (Thành công - Mã 200 OK):** Người dùng nhập đúng cú pháp và dữ liệu cầu thủ họ cần tìm có xuất hiện trong players.db. Hệ thống đóng gói dữ liệu đó bằng jsonify và trả về mã 200 (OK) cho người dùng.

```
return jsonify({
    "message": "ok",
    "cau thu": data
}), 200 # da tim thay
```

5. **lookup.py:**

```python
import argparse
import requests
from tabulate import tabulate
import csv

parser = argparse.ArgumentParser()
parser.add_argument("--name", type=str)
parser.add_argument("--club", type=str)
args = parser.parse_args()

local = "http://127.0.0.1:5000"

def get_data(tieu_chi, ten):
    url = ""
    target = ""
    if tieu_chi == "name":
        url += local + "/get-players/player?name=" + ten
        target = "cau thu"
    if tieu_chi == "club":
        url += local + "/get-teams/team?name=" + ten
        target = "doi bong"

    api_call = requests.get(url)

    if api_call.status_code == 200:
        data = api_call.json()
        if not data:
            print("Khong tim thay du lieu")
            return

        player_list = data.get(target, [])
        if not player_list:
            return

        print(tabulate(player_list, headers="keys", tablefmt="grid"))

        filename = f"{ten}.csv"
        with open(filename, "w", newline="", encoding="utf-8") as f:
            fieldnames = player_list[0].keys()
            write = csv.DictWriter(f, fieldnames=fieldnames)
            write.writeheader()
            write.writerows(player_list)

if args.name:
    get_data("name", args.name)
if args.club:
    get_data("club", args.club)
```

- **Thư viện và công cụ:**
    + **import argparse:** Thư viện argparse dùng để xây dựng giao diện dòng lệnh (CLI - Command Line Interface), giúp chương trình tự động phân tích đối số (arguments) và tùy chọn (options) được truyền vào khi chạy file.

+ **import requests:** Thư viện requests trong Python được sử dụng để gửi yêu cầu HTTP (HTTP requests), cho phép chương trình tương tác với các dịch vụ web, API.
+ **from tabulate import tabulate:** Thư viện tabulate trong Python cung cấp các công cụ như tabulate, giúp tạo bảng từ các cấu trúc dữ liệu như list và list of dictionary.
+ **import csv:** Dùng để đọc và ghi dữ liệu có định dạng Comma Separated Values (CSV).

- **Khởi tạo:**

```
parser = argparse.ArgumentParser()
parser.add_argument("--name", type=str)
parser.add_argument("--club", type=str)
args = parser.parse_args()
```

+ **parser = argparse.ArgumentParser():** Khởi tạo 1 đối tượng (object) làm nhiệm vụ quản lý và phân tích các chuỗi đối số người dùng nhập vào Terminal.
+ **parser.add_argument("--name", type=str) và parser.add_argument("--club", type=str):** Định nghĩa những thông tin mà người dùng có thể nhập. "type=str" dữ liệu nhận được phải có kiểu chuỗi (string). Dấu "--" báo hiệu rằng đây là các tham số tùy chọn (optional), người dùng có thể nhập 1 trong 2 hoặc cả 2, chương trình cũng không sập (crash).
+ **args = parser.parse_args():** Có tác dụng kích hoạt toàn bộ quy trình. Nó đọc toàn bộ những gì người dùng vừa nhập sau cụm "python lookup.py" trên Terminal, phân tích dựa trên quy tắc của 2 câu lệnh trên, rồi đóng gói lại thành 1 biến (args).

- **Quy trình xử lý API:**

```python
local = "http://127.0.0.1:5000"

def get_data(tieu_chi, ten):
    url = ""
    target = ""
    if tieu_chi == "name":
        url += local + "/get-players/player?name=" + ten
        target = "cau thu"
    if tieu_chi == "club":
        url += local + "/get-teams/team?name=" + ten
        target = "doi bong"

    api_call = requests.get(url)

    if api_call.status_code == 200:
        data = api_call.json()
        if not data:
            print("Khong tim thay du lieu")
            return

        player_list = data.get(target, [])
        if not player_list:
            return
```

+ **local = "http://127.0.0.1:5000":** Đường dẫn máy chủ Flask cục bộ (local Flask server).
+ **get_data(tieu_chi, ten):**
    - **Hàm nhận vào 2 tham số tieu_chi và ten** tương ứng với tiêu chí tìm kiếm ("name" hoặc "club") và giá trị tìm kiếm mà người dùng nhập (tên cầu thủ hoặc tên câu lạc bộ).
    - Khởi tạo **url và target rỗng**
    - Tùy theo tieu_chi, hệ thống sẽ sử dụng **cấu trúc rẽ nhánh "if"** để nối chuỗi, tạo ra đường dẫn API hoàn chỉnh (Endpoint).
    - **target:** Đóng vai trò là 1 từ khóa động (Dynamic Key). Vì API trả về cấu trúc JSON khác nhau tùy theo tieu_chi nên target lưu trữ tên trường dữ liệu (key) cần bóc tách ("cau thu" hoặc "doi bong") để tái sử dụng về sau.

- **api_call = requests.get(url):** Sử dụng hàm get() của thư viện requests để gửi 1 yêu cầu HTTP (HTTP request) với phương thức GET tới máy chủ Flask. Lệnh này sẽ tạm dừng chương trình để chờ máy chủ phản hồi.
- **if api_call.status_code == 200:** Kiểm tra mã trạng thái HTTP. Chỉ khi hệ thống trả về 200 (OK), hệ thống mới tiến hành đọc dữ liệu.
- **data = api_call.json():** Thực hiện giải mã (Deserialize) chuỗi văn bản JSON mà máy chủ trả về, ép nó sang kiểu dữ liệu dictionary của Python để dễ dàng thao tác.
- **Kiểm tra kết quả "if not data":** Nếu máy chủ trả về cấu trúc rỗng, in thông báo lỗi và thoát hàm ("return") để tránh gây lỗi hệ thống.
- **player_list = data.get(target, []):** Thay vì lấy dữ liệu trực tiếp bằng data[target] (dễ gây KeyError nếu máy chủ trả về cấu trúc rỗng hoặc mất kết nối), hàm sử dụng phương thức get() kèm theo giá trị dự phóng (Fallback value) là 1 danh sách rỗng "[]". Tức là, nếu chương trình tìm thấy trường dữ liệu có tên khớp với biến động target trong từ điển data, nó sẽ gán trường dữ liệu đó vào player_list. Ngược lại, chương trình trả về danh sách rỗng "[]" để hệ thống không bị crash.
- **Kiểm tra kết quả "if not player_list":** Trường hợp dữ liệu bóc ra bị rỗng thì kết thúc chương trình.

- **Tạo bảng và xuất file CSV**

```
print(tabulate(player_list, headers="keys", tablefmt="grid"))

filename = f"{ten}.csv"
with open(filename, "w", newline="", encoding="utf-8") as f:
    fieldnames = player_list[0].keys()
    write = csv.DictWriter(f, fieldnames=fieldnames)
    write.writeheader()
    write.writerows(player_list)
```

  + **print(tabulate(player_list, headers="keys", tablefmt="grid")):** In ra bảng dạng lưới ô vuông ("tablefmt="grid"") với dữ liệu kiểu từ điển lưu trong player_list, với tên các cột là các key ("headers="key"").

+ **Xuất file CSV:**
  - **filename = f“{ten}.csv”:** Dùng f-string để chèn trực tiếp giá trị của biến ten (tên người dùng nhập) vào chuỗi. Kết quả sẽ tạo 1 file tự động, giúp phân biệt rõ ràng các file kết quả sau mỗi lần tìm kiếm.
  - **with open(filename,“w”,newline=“”,encoding=“utf-8”) as f:**
    - **with open() as f:** Cú pháp Context Manager của Python giúp tự động đóng file sau khi ghi xong, tránh việc quên viết f.close().
    - **“w”:** Chế độ ghi. Nếu file chưa tồn tại, nó sẽ tạo file mới. Nếu file đã tồn tại, nó sẽ ghi đề nội dung mới lên.
    - **newline=“”:** Tham số của open() giúp vô hiệu hóa cơ chế tự động chèn dòng trống, đảm bảo các hàng dữ liệu trong file CSV luôn nằm sát nhau.
    - **encoding=“utf-8”:** Định dạng mã hóa chuẩn quốc tế. Tham số này đảm bảo các kí tự đặc biệt luôn được hiển thị chính xác, tránh lỗi font.
    - **filenames = player_list[0].keys():** Do player_list là 1 mảng chứa các từ điển, câu lệnh này sẽ truy cập phần tử đầu tiên của mảng ([0]) và lấy ra toàn bộ các khóa (keys()). Danh sách các khóa này sẽ được sử dụng làm tiêu đề cho mỗi cột trong bảng CSV.
    - **write = csv.DictWriter(f, fieldnames=fieldnames):** Khởi tạo 1 đối tượng DictWriter từ thư viện csv, có vai trò nhận dữ liệu đầu vào là các từ điển, tự động dò tìm các key và xếp dữ liệu (value) vào đúng cột tương ứng đã được định nghĩa ở fieldnames.
    - **write.writeheader():** Viết dòng đầu tiên vào file, chính là danh sách các tên cột.
    - **write.writerows(player_list):** Hàm writerows() cho phép ghi toàn bộ dữ liệu vào file chỉ với 1 thao tác, thay vì dùng vòng lặp for và hàm writerow() để viết từng dòng.

+ **Quy trình kiểm tra và kích hoạt**
  - **Kiểm tra tham số “--name” (if [args.name](#)):**

        - Câu lệnh này kiểm tra xem thuộc tính "name" bên trong gói args có tồn tại hay không
        - Nếu người dùng không gõ "--name" vào Terminal, biến args.name sẽ mặc định mang giá trị None. Lệnh if sẽ coi None là False và sẽ bỏ qua khối lệnh này.
        - Nếu người có gõ "--name", args.name có dữ liệu (được coi là True) → Chương trình gọi hàm get_data("name", args.name) để kích hoạt các quy trình trên.
    - **Kiểm tra tham số "--club":** Tương tự như khi kiểm tra "--name", hệ thống cũng kiểm tra sự tồn tại của "--club" trong câu lệnh Terminal. Nếu có, nó gọi hàm get_data("club", args.club) để kích hoạt quy trình trên.

6. **controller.py và services.py (thư mục team):**
   Về mặt cấu trúc và logic, 2 file này giống hệt controller.py và services.py trong thư mục players, chỉ khác biệt ở những điểm sau:
   a. **controller.py:**
      - Sử dụng tên phân hệ khác là "team_name", tránh bị trùng với phân hệ cầu thủ ("player_name").
      - Thay đổi định tuyến (route) thành /get-teams/team.
      - Danh sách dữ liệu đội bóng được đóng gói bàng jsonify() được trả về với trường dữ liệu "doi bong" thay vi "cau thu" như ở phân hệ cầu thủ.
   b. **services.py:**
      - Đổi tên hàm lấy dữ liệu từ get_player_data() thành get_team_data().
      - Thay đổi điều kiện lọc trong câu lệnh truy vấn thành query = SELECT FROM players WHERE team = ?" (player đổi thành team).

D. Cách chạy chương trình:

- Tạo ra 2 terminal riêng biệt.
- Trong cả 2 terminal, nhập lệnh "cd restful_api" để truy cập vào thư mục chứa chương trình.
- Tại terminal thứ 1, nhập lệnh "python app.py" để khởi chạy vậy app.py.

- Tại terminal thứ 2, nhập 1 trong 3 lệnh sau:
    + python lookup.py –name “<ten cau thu>”
    + python lookup.py –club “<ten cau lac bo>”
    + python lookup.py –name “<ten cau thu>” –club “<ten cau lac bo>”
- Nếu muốn dừng chương trình, vào terminal đang chạy file app.py, nhập lệnh “Ctrl + C”.

Bài III:

A. Yêu cầu bài tập:

III.

III.1

- Tìm trung vị, trung bình và độ lệch chuẩn của mỗi chỉ số các cầu thủ của mỗi đội. Ghi kết quả vào 1 file CSV.

- Tìm đội bóng có chỉ số điểm số cao nhất ở mỗi chỉ số. Theo bạn đội nào có phong độ tốt nhất giải ngoại Hạng Anh mùa 2024-2025

III.2

- Dựa vào kết quả từ câu I, đề xuất 1 phương pháp định giá cầu thủ.

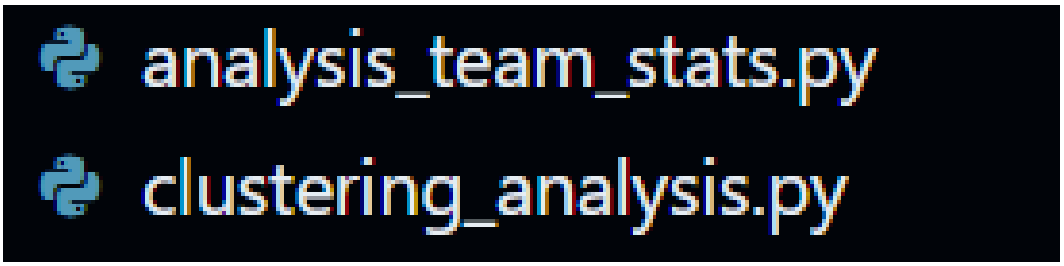
III.3

- Sử dụng thuật toán K-means để phân loại các cầu thủ thành các nhóm có chỉ số tương tự nhau.

- Theo bạn thì nên phân loại cầu thủ thành bao nhiêu nhóm? Vì sao (gợi ý: sử dụng biểu đồ Elbow và Silhoute, vẽ 2 biểu đồ này)? Bạn có Nhận xét gì về kết quả.

- Sử dụng thuật toán PCA, giảm số chiều dữ liệu xuống 2 chiều, 3 chiều, vẽ hình scatter plot phân cụm các điểm dữ liệu trên mặt 2D, và khối 3D.

B. Cấu trúc chương trình:



1. **analysis_team_stats.py**

- Phân tích thống kê theo đội
- Xếp hạng đội bóng

2. **clustering_analysis.py**
   - Phân cụm cầu thủ bằng K-means
   - Đánh giá số cụm
   - Trực quan hóa bằng PCA

C. Giải thích chi tiết:

III.1. Phân tích thống kê theo đội (analysis_team_stats.py) :

1. **Các thư viện:**

```
import sqlite3
import pandas as pd
import numpy as np
from tabulate import tabulate
```

   - **sqlite3:** Dùng để kết nối và truy xuất dữ liệu từ cơ sở dữ liệu SQLite (players.db).
   - **pandas:** Dùng để xử lý và phân tích dữ liệu dạng bảng (DataFrame), hỗ trợ đọc dữ liệu từ database, nhóm dữ liệu và tính toán các thống kê.
   - **numpy:** Hỗ trợ các phép toán số học và xử lý dữ liệu số, đặc biệt trong việc xác định các cột số và tính toán thống kê.
   - **tabulate:** Dùng để hiển thị dữ liệu dưới dạng bảng đẹp trên màn hình console.

2. **Đọc dữ liệu từ database**

```
conn = sqlite3.connect("players.db")
df = pd.read_sql_query("SELECT * FROM players", conn)
conn.close()
```

   - Kết nối SQLite
   - Lấy toàn bộ dữ liệu cầu thủ

3. **Tiền xử lý dữ liệu**
   - Loại bỏ các cột không phải là số: player, team, nationality, position

```
non_numeric = ["player", "team", "nationality", "position"]
```

   - Chuyển dữ liệu sang dạng số:

```
for col in df.columns:
    if col not in non_numeric:
        df[col] = pd.to_numeric(df[col], errors='coerce')
```

   - Dùng pd.to_numeric(errors='coerce')

- Lệnh này quan trọng vì:
  - Dữ liệu crawl là dạng string
  - Phải convert sang số để tính toán
- Xóa cột rỗng:

```
df = df.dropna(axis=1, how='all')
```

- Lấy cột số:

```
numeric_cols = df.select_dtypes(include=np.number).columns
```

4. **Thống kê theo đội (median, mean, std)**
   - **Tính các giá trị:**

```
team_stats = df.groupby("team")[numeric_cols].agg(['median', 'mean', 'std'])
```

     - **Mean** (trung bình) là giá trị trung bình cộng của các chỉ số, phản ánh mức độ chung của toàn đội.
     - **Median** (trung vị) là giá trị nằm giữa khi sắp xếp dữ liệu, giúp giảm ảnh hưởng của các giá trị bất thường (outlier).
     - **Standard Deviation – Std** (độ lệch chuẩn) đo mức độ phân tán của dữ liệu quanh giá trị trung bình; giá trị càng nhỏ thì dữ liệu càng ổn định, càng lớn thì sự biến động càng cao.
     - Trong bài toán, ba đại lượng này giúp đánh giá:
       - Hiệu suất tổng thể (mean)
       - Mức điển hình (median)
       - Độ ổn định phong độ của cầu thủ trong đội (std)
   - **Làm phẳng tên cột :**

```
team_stats.columns = ['_'.join(col) for col in team_stats.columns]
```

   - **Xuất file CSV :**

```
team_stats.to_csv("team_statistics.csv", index=True)
```

→ Lưu vào file: team_statistics.csv

5. **Tìm đội tốt nhất theo từng chỉ số:**

```
best_teams = {}

for col in numeric_cols:
    if col in ["minutes", "games", "goals", "assists"]:
        series = df.groupby("team")[col].sum()
    else:
        series = df.groupby("team")[col].mean()

    if col in ["cards_red", "cards_yellow"]:
        best_teams[col] = series.idxmin()
    else:
        best_teams[col] = series.idxmax()
```

- Với mỗi chỉ số:
  - Nếu là chỉ số cộng dồn (goals, assists, minutes): dùng sum
  - Nếu là chỉ số hiệu suất: dùng mean
- Với chỉ số xấu:
  - cards_red
  - cards_yellow
    - chọn giá trị nhỏ nhất
- Với chỉ số tốt:
  - ➢ chọn giá trị lớn nhất

➢ Lưu kết quả vào file: best_team_each_metric.csv

```
best_df = pd.DataFrame(list(best_teams.items()), columns=["Chi_so", "Doi_tot_nhat"])
best_df.to_csv("best_team_each_metric.csv", index=False)
```

6. Tính đội mạnh nhất tổng thể :

- Chuẩn hóa dữ liệu:

```
df_norm = df.copy()
df_norm[numeric_cols] = (df[numeric_cols] - df[numeric_cols].mean()) / df[numeric_cols].std()
df_norm = df_norm.fillna(0)
```

  - Dùng Z-score normalization là phương pháp biến đổi dữ liệu về cùng một thang đo, dựa trên trung bình (mean) và độ lệch chuẩn (std).
  - Sau khi chuẩn hóa:
    - Dữ liệu có mean = 0
    - Độ lệch chuẩn = 1
  - Giúp các chỉ số khác nhau (goals, assists, minutes…) có thể so sánh công bằng
- Đảo dấu chỉ số xấu :

```
negative_cols = ["cards_red", "cards_yellow"]
for col in negative_cols:
    if col in df_norm.columns:
        df_norm[col] *= -1
```

- thẻ đỏ, thẻ vàng → nhân -1

- **Gán trọng số:**

```
weights = {
    "goals": 2,
    "assists": 1.5,
    "goals_per90": 2,
    "cards_red": -2,
    "cards_yellow": -1
}
```

- **Tính điểm:**
  - Trung bình từng đội × trọng số

```
team_score = pd.Series(0, index=df["team"].unique())

for col in numeric_cols:
    w = weights.get(col, 1)
    team_score += df_norm.groupby("team")[col].mean() * w
```

- **Đội mạnh nhất:**

```
best_team_overall = team_score.idxmax()
```

  - mỗi chỉ số được gán trọng số theo mức độ quan trọng và tính điểm trung bình cho từng đội. Tổng điểm của đội được tính bằng cách cộng các chỉ số đã nhân trọng số, từ đó xác định đội có điểm cao nhất là đội có phong độ tốt nhất.

- **Kết quả:**
  - File CSV:
    - team_statistics.csv
    - best_team_each_metric.csv
  - In ra:
    - bảng thống kê
    - bảng xếp hạng
    - đội tốt nhất

III.2. Phương pháp định giá cầu thủ

- Đề xuất: Giá cầu thủ có thể được tính theo công thức:

  Giá trị ≈ (Hiệu suất thi đấu × Trọng số) + (Độ ổn định)

- Các yếu tố quan trọng:
  - Bàn thắng (goals)
  - Kiến tạo (assists)
  - Hiệu suất per90
  - Thời gian thi đấu
  - Kỷ luật (thẻ)
- Cách làm cụ thể:
  - Chuẩn hóa các chỉ số
  - Gán trọng số
  - Tính tổng điểm
  - So sánh với giá thực tế
- Nhận xét
  - Cầu thủ tấn công thường giá cao hơn
  - Cầu thủ có hiệu suất cao nhưng ít phút có thể bị đánh giá thấp

III.3. Phân cụm cầu thủ (clustering_analysis.py)

1. **Chuẩn bị dữ liệu:**

```
import sqlite3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
```

- **matplotlib.pyplot:** Dùng để vẽ biểu đồ như Elbow Method, Silhouette Score và scatter plot PCA.
- **StandardScaler (sklearn.preprocessing):** Dùng để chuẩn hóa dữ liệu theo Z-score, giúp các đặc trưng có cùng thang đo.
- **KMeans (sklearn.cluster):** Thuật toán dùng để phân cụm các cầu thủ thành các nhóm có đặc điểm tương tự.
- **silhouette_score (sklearn.metrics):** Dùng để đánh giá chất lượng phân cụm và hỗ trợ chọn số cụm k tối ưu.
- **PCA (sklearn.decomposition):** Dùng để giảm số chiều dữ liệu và trực quan hóa kết quả phân cụm (2D, 3D).

2. **Chuẩn bị dữ liệu:**

```
non_numeric = ["player", "team", "nationality", "position"]

df_num = df.drop(columns=non_numeric)
df_num = df_num.apply(pd.to_numeric, errors='coerce')
df_num = df_num.dropna()
```

- Loại bỏ cột không số
- Chuyển về numeric
- Xóa dòng thiếu dữ liệu

3. **Chuẩn hóa dữ liệu**

```
scaler = StandardScaler()
X = scaler.fit_transform(df_num)
```

- StandardScaler là công cụ trong thư viện scikit-learn dùng để chuẩn hóa dữ liệu về cùng thang đo (mean = 0, std = 1) giúp mô hình học chính xác hơn.

4. **Elbow Method**

```
inertia = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
```

- Elbow Method là phương pháp dùng để xác định số cụm (k) phù hợp trong thuật toán K-means.
- Dùng để chọn số cụm k bằng cách vẽ inertia theo k và chọn điểm "khuỷu tay", nơi việc tăng k không còn cải thiện nhiều.

- **Vẽ biểu đồ:**

```
plt.figure()
plt.plot(K_range, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Số cụm (k)")
plt.ylabel("Inertia")
plt.show()
```

5. **Silhouette Score**

```
sil_scores = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)
    sil_scores.append(silhouette_score(X, labels))
```

- Silhouette Score dùng để đánh giá chất lượng phân cụm, giá trị càng gần 1 thì phân cụm càng tốt. Số cụm tối ưu là giá trị k có Silhouette Score cao nhất.

- **Vẽ biểu đồ:**

```
plt.figure()
plt.plot(K_range, sil_scores, marker='o')
plt.title("Silhouette Score")
plt.xlabel("Số cụm (k)")
plt.ylabel("Score")
plt.show()
```

- Số cụm k được chọn dựa trên Elbow Method (điểm khuỷu tay) và Silhouette Score (giá trị cao nhất). Trong bài, chọn k=4 vì phù hợp cả hai tiêu chí.

6. **Phân cụm K-means:**

```
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X)
```

- Thuật toán K-means được dùng để gán mỗi cầu thủ vào một cụm tương ứng, và kết quả được lưu vào file players_clustered.csv.

7. **PCA 2D:**

- Giảm dữ liệu xuống 2 chiều để vẽ biểu đồ scatter, mỗi màu là một cụm.

```
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X)
```

- Vẽ scatter:

```
plt.figure(figsize=(12, 8))
scatter = plt.scatter(X_2d[:, 0], X_2d[:, 1], c=df["cluster"], cmap='viridis', alpha=0.6)
plt.colorbar(scatter, label='Cluster ID')
```

- Hiển thị tên cầu thủ: chọn ra ngẫu nhiên 15 cầu thủ để hiển thị

```
import random
random_indices = random.sample(range(len(df)), min(15, len(df)))
for i in random_indices:
    plt.annotate(df['player'].iloc[i],
                 (X_2d[i, 0], X_2d[i, 1]), fontsize=8, alpha=0.9, xytext=(5, 5),textcoords='offset points')
```

8. PCA 3D:

- Giảm xuống 3 chiều để trực quan hóa dữ liệu trong không gian 3D rõ hơn.

```
pca_3d = PCA(n_components=3)
X_3d = pca_3d.fit_transform(X)
```

- Vẽ:

```
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
p = ax.scatter(X_3d[:, 0], X_3d[:, 1], X_3d[:, 2], c=df["cluster"], cmap='viridis', s=50, alpha=0.6)
fig.colorbar(p, ax=ax, label='Cluster ID', shrink=0.5)
```

9. Lưu kết quả vào players_clustered.csv

```
df.to_csv("players_clustered.csv", index=False)
```

10. Ý nghĩa các nhóm:

- Nhóm 1: cầu thủ tấn công mạnh (goals, assists cao)
- Nhóm 2: cầu thủ trung bình
- Nhóm 3: ít thi đấu
- Nhóm 4: thiên về phòng ngự